

Plano de Leitura e Desenvolvimento — Tarefa 1

O Início da Operação

Visão Geral do Dia

O dia está dividido em **5 blocos**. Cada bloco tem: o que ler, o conceito central, e o que implementar logo em seguida.

BLOCO 1 Objects First Cap. 1	BLOCO 2 Objects First Cap. 2	BLOCO 3 Objects First Cap. 3	BLOCO 4 Dean & Dean Cap. 3 + 7	BLOCO 5 Building Maintainable Guideline 4
↓	↓	↓	↓	↓
Entender o que são objetos e classes	Escrever MateriaPrima e Produto	Escrever Maquina e Esteira e EstacaoInspecao	Escrever Main (Scanner + menu + array de Produtos)	Refatorar o código com DRY (eliminar repetição)

Bloco 1 — Fundação: O que são objetos e classes

Livro: *Objects First with Java* — Capítulo 1 **Tempo sugerido de leitura:** 30–40 min

O que aprender neste capítulo

- O que é um **objeto** (tem estado e comportamento)
- O que é uma **classe** (é o molde do objeto)
- Como **criar** um objeto a partir de uma classe (`new`)
- Como **chamar métodos** de um objeto

Como isso aparece no projeto

Conceito do Cap. 1	Onde aparece na Tarefa
Criar objeto com <code>new</code>	<code>new MateriaPrima(...)</code> , <code>new Maquina(...)</code> , <code>new Produto(...)</code>
Chamar método	<code>maquina.ligar()</code> , <code>esteira.adicionarItem(mp)</code> , <code>estacao.ativar()</code>
Objeto tem estado	<code>maquina.ligada</code> , <code>esteira.emMovimento</code> , <code>estacao.ativa</code>
Classe é o molde	Cada arquivo <code>.java</code> é uma classe (molde)

Após ler: exercício mental

Abra o código de `Main.java`. Identifique **todas** as linhas onde um objeto é criado com `new` e todas onde um método é chamado com `.`. Não escreva nada ainda — só observe.

Bloco 2 — Primeiras classes: `MateriaPrima` e `Produto`

Livro: *Objects First with Java* — Capítulo 2 **Tempo sugerido de leitura:** 40–50 min

O que aprender neste capítulo

- Como declarar **atributos** (`private int id`, `private String nome`)
- Como escrever um **construtor** (inicializa o objeto)
- Como escrever **getters** (acessam atributos privados)
- Como escrever **setters/mutadores** (alteram atributos)
- O que é **encapsulamento** (atributo privado + método público)

Como isso aparece no projeto

Conceito do Cap. 2	Onde aparece na Tarefa
Atributos <code>private</code>	Todos os atributos de <code>MateriaPrima</code> e <code>Produto</code>
Construtor com parâmetros	<code>MateriaPrima(int id, String nome, double qtd, ...)</code>
Getters	<code>getNome()</code> , <code>getQuantidade()</code> , <code>getStatus()</code>
Encapsulamento	Nenhuma classe muda <code>quantidade</code> diretamente — só via <code>consumir()</code>

Após ler: o que implementar

1. Escreva `MateriaPrima.java` do zero (sem olhar o guia)
2. Escreva `Produto.java` do zero
3. Compare com o guia e corrija diferenças
4. Verifique: todos os atributos estão `private`? Todos os getters estão `public`?

Obrigações desta etapa (checklist)

- [] `MateriaPrima` tem atributos: `id`, `nome`, `quantidade`, `unidade`, `quantidadeMinima`
- [] `MateriaPrima` tem métodos: `consumir()`, `adicionarEstoque()`, `verificarDisponibilidade()`, getters
- [] `Produto` tem atributos: `id`, `nome`, `status`, `quantidadeMateriaPrimaNecessaria`
- [] `Produto` tem métodos: `processar()`, `definirDemandaMateriaPrima()`, `getDemandaMateriaPrima()`, getters
- [] `status` de `Produto` começa como `"AGUARDANDO_PROCESSAMENTO"` sem que o usuário defina
- [] Nome do produto é definido no código, **não digitado pelo usuário**

Bloco 3 — Comportamento e interação: Máquina, Esteira, EstacaoInspecao

Livro: *Objects First with Java* — Capítulo 3 **Tempo sugerido de leitura:** 40–50 min

O que aprender neste capítulo

- Como objetos **interagem** entre si (um chama métodos do outro)
- O que é **estado interno** e como ele controla o comportamento
- Como **passar objetos como parâmetros** para métodos
- O conceito de **composição** (objeto contém referência a outro objeto)

Como isso aparece no projeto

Conceito do Cap. 3	Onde aparece na Tarefa
Estado interno controla comportamento	Máquina só processa se <code>ligada == true</code>
Objeto como parâmetro	<code>maquina.processar(mp, demanda, produto)</code>
Interação entre objetos	<code>estacao.inspecionar(produto)</code> chama <code>produto.inspecionar()</code> internamente
Composição	Esteira armazena referência a <code>MateriaPrima</code> ou <code>Produto</code> no atributo <code>item</code>

Após ler: o que implementar

1. Escreva `Maquina.java` do zero
2. Escreva `Esteira.java` do zero
3. Escreva `EstacaoInspecao.java` do zero
4. Compare com o guia e corrija

Obrigações desta etapa (checklist)

- [] `Maquina` tem atributos: `nome`, `ligada`, `capacidadeMaxima`
- [] `Maquina` tem métodos: `ligar()`, `desligar()`, `processar()`, `getNome()`, `estaLigada()`
- [] `processar()` da `Maquina` **bloqueia** se máquina estiver desligada
- [] `processar()` **bloqueia** se demanda exceder `capacidadeMaxima`
- [] `processar()` **bloqueia** se matéria-prima for insuficiente
- [] `Esteira` tem atributos: `item`, `emMovimento`, `capacidadeMaxima`
- [] `Esteira` tem métodos: `ligar()`, `desligar()`, `adicionarItem()`, `removerItem()`, `verificarCapacidade()`
- [] `adicionarItem()` **bloqueia** se esteira estiver parada
- [] `adicionarItem()` **bloqueia** se a esteira já tiver um item
- [] `EstacaoInspecao` tem atributos: `ativa`, `produtosInspeccionados`
- [] `EstacaoInspecao` tem métodos: `ativar()`, `desativar()`, `inspecionar()`, `getTotalInspeccionados()`
- [] `inspecionar()` **bloqueia** se estação estiver desativada

Bloco 4 — Entrada do usuário e menu: Main

Livro: *Introduction to Programming with Java* — Dean & Dean, Capítulos 3 e 7 **Tempo sugerido de leitura:** 40 min (Cap. 3: Scanner e controle de fluxo | Cap. 7: arrays)

O que aprender nestes capítulos

- Como usar o `Scanner` para ler entradas numéricas do terminal
- Como usar `while`, `switch` e `if-else` para criar menus
- Como declarar e percorrer um **array de objetos** (`Produto[]`)

Como isso aparece no projeto

Conceito do Cap. 3 e 7	Onde aparece na Tarefa
<code>Scanner sc = new Scanner(System.in)</code>	Leitura de opções numéricas do usuário
<code>while (opcao != 0)</code>	Loop do menu principal
<code>switch (opcao)</code>	Direciona para cada opção do menu
<code>Produto[] produtos = new Produto[3]</code>	Array com os produtos pré-definidos
Percorrer array com <code>for</code>	Exibir lista de produtos disponíveis

Após ler: o que implementar

1. Escreva `Main.java` do zero
2. O menu deve ter opções numéricas (1, 2, 3, 0)
3. Implemente o método `lerInteiro()` para validar entrada (DRY)
4. Compare com o guia e corrija

Obrigações desta etapa (checklist — sequência obrigatória do enunciado)

- [] 1. Instanciar ao menos uma `MateriaPrima` com estoque inicial
- [] 2. Instanciar produtos pré-definidos no código (array de `Produto`)
- [] 3. Instanciar uma `Maquina`
- [] 4. Instanciar uma `Esteira`
- [] 5. Instanciar uma `EstacaoInspecao`
- [] 6. Exibir menu com opções **numéricas**
- [] 7. Usuário seleciona o produto via número
- [] 8. Usuário define a demanda de matéria-prima via número
- [] 9. Verificar se há matéria-prima suficiente antes de produzir
- [] 10. Ligar os equipamentos necessários
- [] 11. Colocar matéria-prima na esteira
- [] 12. Transportar matéria-prima até a máquina
- [] 13. Processar na máquina com a demanda definida
- [] 14. Transportar produto via esteira até a estação de inspeção
- [] 15. Realizar inspeção do produto
- [] 16. Permitir consulta ao estoque de matéria-prima
- [] 17. Informar o status da produção ao longo da execução

- [] **Scanner aceita APENAS entradas numéricas** (nenhum `nextLine()` de nome pelo usuário)
- [] Nomes de produtos, MP e máquinas definidos no código-fonte

Bloco 5 — Refinamento: DRY e boas práticas

Livro: *Building Maintainable Software* — Guideline 4: Write Code Once **Tempo sugerido de leitura:** 20 min

O que aprender nesta seção

- O que é **DRY** (Don't Repeat Yourself): lógica duplicada = risco de bug
- Como identificar código repetido e extraí-lo para um método único
- Por que usar **constantes** (`static final`) em vez de strings mágicas

Como isso aparece no projeto

Conceito DRY	Onde aplicar na Tarefa
Método <code>lerInteiro()</code> em <code>Main</code>	Em vez de repetir <code>scanner.nextInt()</code> + validação em 3 lugares
<code>verificarDisponibilidade()</code> em <code>MateriaPrima</code>	Usado em <code>consumir()</code> E em <code>Main</code> — lógica escrita uma só vez
Constantes <code>STATUS_AGUARDANDO</code> , <code>STATUS_PROCESSADO</code>	Em vez de escrever a string <code>"AGUARDANDO_PROCESSAMENTO"</code> em vários lugares

Após ler: o que revisar

1. Existe alguma verificação de nulo ou de estado escrita mais de uma vez? → Extraia para método
2. Existe algum texto/string repetido? → Vire constante `static final`
3. Existe algum trecho de código copiado e colado? → Vire método

Mapa Completo: Leitura → Código → Obrigação do Enunciado

Bloco	Capítulo	Classe produzida	Obrigação do enunciado atendida
1	Objects First Cap. 1	— (só leitura)	Entendimento base de objetos
2	Objects First Cap. 2	<code>MateriaPrima</code> , <code>Produto</code>	Itens 1, 2 do Main; estados de MP e Produto
3	Objects First Cap. 3	<code>Maquina</code> , <code>Esteira</code> , <code>EstacaoInspecao</code>	Itens 3, 4, 5 do Main; restrições de estado
4	Dean & Dean Cap. 3 + 7	<code>Main</code>	Itens 6 ao 17 do Main; Scanner numérico
5	Building Maintainable Guideline 4	Revisão geral	DRY, constantes, sem duplicação

Checklist Final de Entrega

Arquivos a entregar

- ☐ `MateriaPrima.java`
- ☐ `Produto.java`
- ☐ `Maquina.java`
- ☐ `Esteira.java`
- ☐ `EstacaoInspecao.java`
- ☐ `Main.java`

Requisitos técnicos obrigatórios

- ☐ Todos os atributos são `private`
- ☐ Acesso aos atributos feito apenas por métodos `public`
- ☐ Nenhuma herança (`extends`) ou classe abstrata usada
- ☐ `Scanner` aceita **apenas números** do usuário
- ☐ Nomes de produtos, MP e máquinas estão no código-fonte (não digitados pelo usuário)
- ☐ Máquina desligada não processa
- ☐ Esteira parada não transporta e não aceita novo item se já tiver um
- ☐ Estação desativada não inspeciona
- ☐ Estoque de MP é atualizado após cada produção
- ☐ Usuário é informado se o estoque for insuficiente
- ☐ Fluxo de produção segue a sequência: MP → Esteira → Máquina → Esteira → Inspeção
- ☐ Status final do produto é exibido ao usuário

Conceitos POO a demonstrar (para o professor)

- ☐ **Encapsulamento** — atributos `private` + getters/setters
- ☐ **Composição** — `Maquina` usa `MateriaPrima` e `Produto` via parâmetro
- ☐ **Coesão** — cada classe tem uma única responsabilidade
- ☐ **DRY** — sem lógica duplicada
- ☐ **KISS** — sem herança, GUI ou banco de dados

Plano elaborado com base no enunciado completo da Tarefa 1 — O Início da Operação.